

Software Design Description 100

ESP32-CAM – Overview

Overview

The ESP32-CAM subsystem captures grayscale images from an OV2640 sensor, forwards raw frames to the ESP32-DevKitV1 over SPI DMA, and periodically saves JPEG snapshots to an SD card. A burst capture test mode dumps raw frames to SD for offline simulation analysis.

Hardware: ESP32-D0WD (ESP32-CAM board) | 8 MB PSRAM | OV2640 camera | SD card (4-bit SDMMC) | SPI master to DevKitV1

Module Hierarchy

main.c	Entry point - mode selection, init sequence
---- camera_control.c	Capture loop (production) / burst task (test)
---- ov2640_config.c	Sensor configuration, proportional AEC auto-tune
---- DMA_SPI_master.c	SPI DMA master - sends frames to DevKitV1
---- image_buffer_pool.c	PSRAM buffer pool (3 x 76.8 KB, thread-safe)
\---- sd_storage.c	SD card mount/unmount, JPEG and raw file writes

Operating Modes

Production Mode (default)

- Camera captures at 30 fps target (33 ms period)
- Every frame: raw grayscale enqueued to SPI transmit task via FreeRTOS queue
- Every 10th frame: JPEG encoded and saved to SD card
- SPI and SD share GPIO13 — time-multiplexed in the capture loop (SPI deinit → SD mount → write → unmount → SPI reinit)

Burst Capture Mode (`#define TEST_BURST_CAPTURE`)

- SD card mounted once at boot, stays mounted
 - Discards first 5 frames (AEC settle time)
 - Captures 30 raw grayscale frames (320x240, 76.8 KB each) at max rate
 - Saves to `/sdcard/img_NNNN.raw`
 - No SPI, no buffer pool — dedicated SD path
 - Used for harvesting simulation data (fed into `sim/wht_sim.py`)
-

SPI DMA Interface

Parameter	Value
Host	HSPI_HOST
Clock	10 MHz
Mode	0 (CPOL=0, CPHA=0)
MOSI	GPIO 13
MISO	GPIO 12
SCLK	GPIO 14
CS	GPIO 15
Max transfer	4092 bytes (DMA limit)

Parameter	Value
DMA channel	Auto

Frame protocol: Each frame transmission starts with a 12-byte header:

Field	Size	Value
Magic	4 bytes	0xCAFEDEED
Size	4 bytes	Frame size in bytes
Checksum	4 bytes	XOR of all image bytes

Followed by the raw pixel data in 4092-byte DMA chunks.

Bus mutex: `spi_dma_transmit()` holds a mutex for the full duration of each transfer. `spi_dma_deinit()` blocks on this mutex, ensuring no in-flight transactions when tearing down the bus for SD card access.

OV2640 Configuration

Parameter	Value
Resolution (production)	640x480 grayscale
Resolution (burst)	320x240 grayscale
Pixel format	GRAYSCALE
Auto exposure	Disabled after auto-tune
Auto gain	Disabled (gain=0)
Auto white balance	Disabled

Proportional AEC auto-tune: On init, `ov2640_config.c` runs a closed-loop exposure tuner: 1. Set initial AEC value (default 50) 2. Capture frame, compute mean pixel brightness 3. Adjust: `aec_next = aec_current * target / mean` 4. Clamp to [1, 1200], repeat until `|mean - target| < tolerance` 5. Converges in 3-5 iterations typically

Target mean brightness: 110. Tolerance: 10.

Known limitation: In full direct sunlight, minimum AEC (`aec=1`) still overexposes. The sensor bottoms out and cannot reduce exposure further. Deploy with shade visor or recessed housing.

PSRAM Buffer Pool

- 3 buffers x 76.8 KB allocated in PSRAM at boot
- Thread-safe alloc/free via FreeRTOS mutex
- Peak usage tracking for diagnostics
- Used by production mode only (SPI transmit task borrows a buffer, capture task fills another)

SD Card

- 4-bit SDMMC mode at 40 MHz (high-speed)
- Mount point: `/sdcard`
- Auto-incrementing filename counter (`img_0000.jpg`, `img_0001.raw`, etc.)
- Production: mount/unmount per write (pin sharing with SPI)
- Burst mode: mount once, stay mounted

Source Files

File	Purpose	Status
<code>src/main.c</code>	Entry point, mode selection	Done
<code>src/camera_control.c</code>	Capture loop + burst task	Done
<code>src/ov2640_config.c</code>	Sensor config, AEC auto-tune	Done
<code>src/DMA_SPI_master.c</code>	SPI DMA master, frame protocol	Done
<code>src/image_buffer_pool.c</code>	PSRAM buffer pool	Done
<code>src/sd_storage.c</code>	SD mount/write/unmount	Done

References

- SDD000 — Development Standards
- SDD200 — ESP32-DevKitV1 (SPI slave receiver)
- ESP-IDF Camera Driver